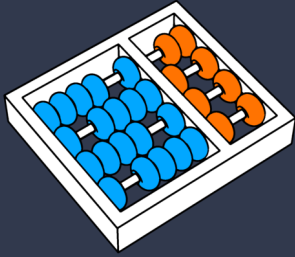




UNICAMP



Langchain e Integrações

Prof. Dr. André Gomes Regino

regino@unicamp.br

Agenda



01. Contexto

LangChain como camada de integração



02. Provedores de LLM

Integrações com principais modelos



03. Bancos Vetoriais

FAISS, Chroma e Pinecone



04. Fontes de Dados

PDF, CSV e consultas SQL



05. Ferramentas Externas

Uso de Tools e Agentes



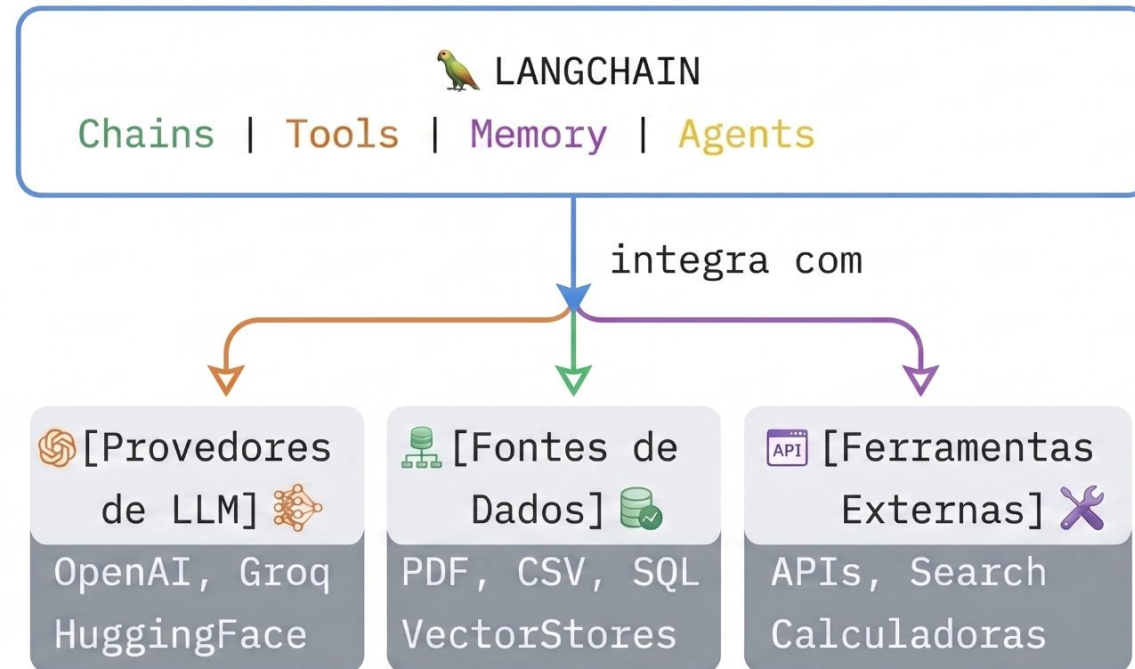
06. Deploy e Interfaces

Streamlit e FastAPI

Contexto e Objetivos

Contexto

- LangChain orchestra, mas precisa de parceiros



como o LangChain se conecta a cada um desses componentes externos? 4

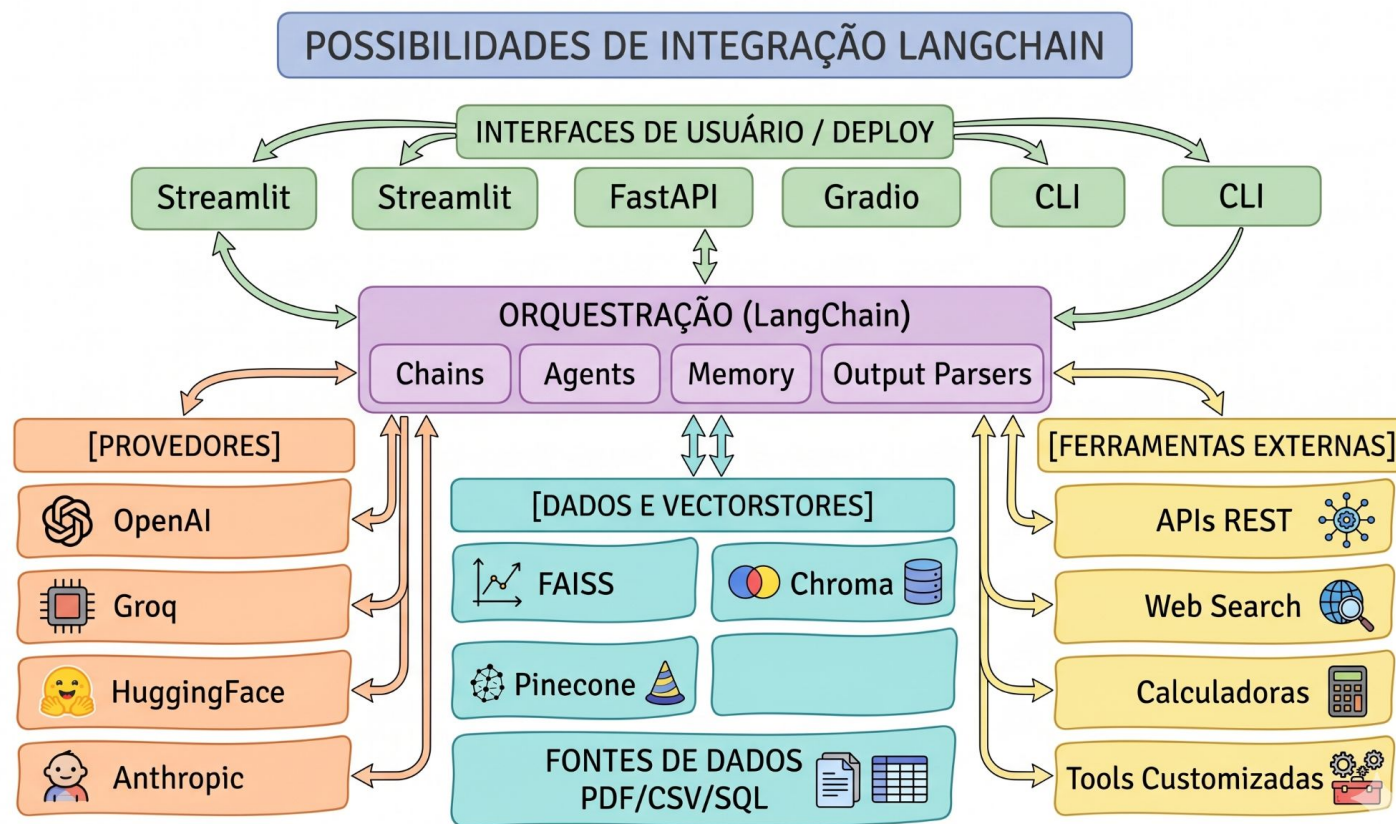
Contexto

LangChain como camada de integração: o que isso significa na prática

Sem LangChain	Com LangChain
Código específico por provedor de LLM	Interface <code>invoke()</code> igual para todos
Parsing manual de documentos	<code>DocumentLoader</code> padronizado
Deploy manual do modelo	Encapsula em uma <code>tool/chain/API</code>

A abstração tem um custo (complexidade, versioning), mas o ganho é portabilidade: trocar de OpenAI para Groq é mudar uma linha de código.

Arquitetura Expandida



Objetivos

- **Trocar** provedores de LLM alterando poucas linhas de código
- **Entender** como criar uma base de dados vetorial para busca
- **Criar** ferramentas para dar acesso ao mundo exterior ao LLM
- **Entender** como estruturar uma API REST básica

Integrações com LLMs

Provedores de LLM



OpenAI



groq



Hugging Face

Por que Múltiplos Provedores

- Custo
- Latência
- Modelos diferentes
- trocar backend sem mudar sistema

Por que Múltiplos Provedores

Como decidir qual provedor usar

Critério	Melhor opção
Menor custo por token	HuggingFace (self-hosted) ou Groq
Menor latência de resposta	Groq (hardware LPU dedicado)
Melhor qualidade em tarefas complexas	?
Dados sensíveis / privacidade	HuggingFace local (sem envio externo)
Sem acesso à internet (on-premise)	HuggingFace + Ollama

Múltiplos Provedores

```
1 # Instalação
2 # Versão dos pacotes usados neste exemplo: langchain-openai>=0.2, langchain-
  groq>=0.1, langchain-huggingface>=0.1
3 # pip install langchain-openai langchain-groq langchain-huggingface
4
5 from langchain_openai import ChatOpenAI
6 from langchain_groq import ChatGroq
7 from langchain_huggingface import HuggingFaceEndpoint
8
9 # OpenAI – padrão de mercado
10 llm_openai = ChatOpenAI(model="gpt-4o-mini", temperature=0)
11
12 # Groq – ultra-rápido, ideal para prototipagem
13 llm_groq = ChatGroq(model="llama-3.3-70b-versatile", temperature=0)
14
15 # HuggingFace – open-source, via Inference API
16 llm_hf = HuggingFaceEndpoint(
17     repo_id="google/flan-t5-base",
18     temperature=0.5
19 )
20
21 # Interface IDÊNTICA para todos:
22 pergunta = "Explique IA em uma frase."
23 print(llm_openai.invoke(pergunta).content)
24 print(llm_groq.invoke(pergunta).content)
25 print(llm_hf.invoke(pergunta))
```

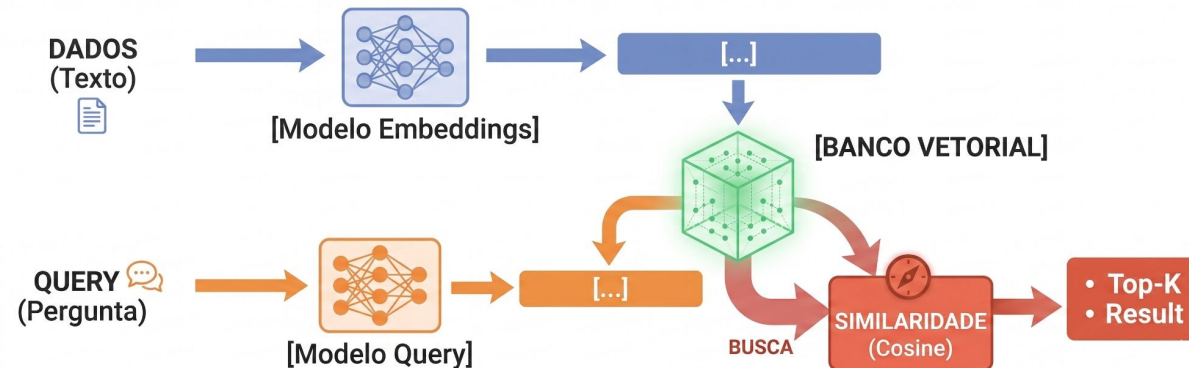
Bancos Vetoriais

Um LLM não "lembra" de documentos que ele não viu no treinamento

O que são Bancos Vetoriais

Principais integrações

- FAISS*
- Chroma
- Pinecone



*JOHNSON, Jeff; DOUZE, Matthijs; JÉGOU, Hervé. Billion-scale similarity search with GPUs. **IEEE transactions on big data**, v. 7, n. 3, p. 535-547, 2019.

Exemplo

```
1 # Versão: langchain-community>=0.2, langchain-openai>=0.2, faiss-cpu>=1.7,  
  chromadb>=0.5  
2  
3 from langchain_openai import OpenAIEmbeddings  
4 from langchain_community.vectorstores import FAISS, Chroma  
5 from langchain_core.documents import Document  
6  
7 # Documentos de exemplo  
8 docs = [  
9     Document(page_content="LangChain é um framework de orquestração de LLMs."),  
10    Document(page_content="FAISS é uma biblioteca de busca vetorial da Meta."),  
11    Document(page_content="RAG combina recuperação e geração de texto."),  
12 ]  
13  
14 embeddings = OpenAIEmbeddings(model="text-embedding-3-small")  
15  
16 # FAISS – local, em memória  
17 faiss_store = FAISS.from_documents(docs, embeddings)  
18 faiss_store.save_local("./meu_indice") # persiste no disco  
19  
20 # Chroma – local, com persistência SQLite  
21 chroma_store = Chroma.from_documents(  
22     docs, embeddings, persist_directory="./chroma_db"  
23 )  
24  
25 # Busca semântica – mesma interface para ambos  
26 query = "O que é recuperação de informação?"  
27 resultados = faiss_store.similarity_search(query, k=2)  
28 for r in resultados:  
29     print(r.page_content)
```

Instabilidade de API: Realidade LangChain

Evolução das Versões

< 0.1

Pacote monolítico: `langchain.*`

0.1 – 0.2

Divisão em `community` e `core`

0.3+

Integrações (OpenAI, Groq) separadas

Exemplo de Mudança de Código



```
# API antiga (< 0.1) – Quebra o código  
from langchain.chat_models import ChatOpenAI
```



```
# API atual (>= 0.1)  
from langchain_openai import ChatOpenAI
```

Nota: Os conceitos e o racional desta aula são estáveis. O código é um detalhe de implementação que muda rapidamente.

Integrações com Dados

Para responder com os seus dados, é preciso carregá-los e conectá-los ao pipeline

Integrações Suportadas

- PDFs
- CSV
- SQL
- TXT

```
1 # Versão: langchain-community>=0.2
2
3 from langchain_community.document_loaders import (
4     PyPDFLoader,
5     CSVLoader,
6     TextLoader,
7 )
8 from langchain_community.utilities import SQLDatabase
9
10 # Carregar PDF
11 loader_pdf = PyPDFLoader("artigo.pdf")
12 docs_pdf = loader_pdf.load()
13
14 # Carregar CSV
15 loader_csv = CSVLoader("dados.csv", encoding="utf-8")
16 docs_csv = loader_csv.load()
17
18 # Conectar ao banco SQL
19 db = SQLDatabase.from_uri("sqlite:///meubanco.db")
20 print(db.get_usable_table_names()) # inspeciona as tabelas disponíveis
```

Ferramentas Externas

O que são Tools

Funções que o LLM pode decidir chamar dinamicamente
qualquer função Python pode virar uma tool com o decorator `@tool`

Tool	O que faz	Pacote
DuckDuckGoSearchRun	Busca na web sem API key	<code>duckduckgo-search</code>
WikipediaQueryRun	Consulta Wikipedia	<code>wikipedia</code>
PythonREPLTool	Executa código Python	<code>langchain-experimental</code>
SQLDatabaseTool	Consulta banco SQL natural	<code>langchain-community</code>
RequestsGetTool	Faz chamadas HTTP GET	<code>langchain-community</code>

Exemplo

```
1 # Versão: langchain-core>=0.3
2 from langchain_core.tools import tool
3 from langchain_openai import ChatOpenAI
4
5 # Tool customizada, a docstring é o que o LLM lê para decidir quando usar
6 @tool
7 def consultar_estoque(produto: str) -> str:
8     """Consulta o estoque atual de um produto pelo nome.
9     Use esta tool quando o usuário perguntar sobre disponibilidade."""
10    estoque = {"notebook": 5, "mouse": 23, "teclado": 0}
11    qtd = estoque.get(produto.lower(), -1)
12    if qtd == -1:
13        return f"Produto '{produto}' não encontrado no estoque."
14    return f"Estoque de '{produto}': {qtd} unidades."
15
16 # Binding da tool ao LLM
17 llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
18 llm_com_tool = llm.bind_tools([consultar_estoque])
19
20 # O LLM decide chamar a tool automaticamente
21 resposta = llm_com_tool.invoke("Tem notebook disponível?")
22 print(resposta.tool_calls)  # {'name': 'consultar_estoque', 'args': {'produto': 'notebook'}}
```

Deploy e Interface

Interface com Streamlit

- rápida
- simples
- ideal para demo

```
1 # app.py – execute com: streamlit run app.py. Não funciona no Google colab!
2 import streamlit as st
3 from langchain_openai import ChatOpenAI
4 from langchain_core.messages import HumanMessage, AIMessage
5
6 st.title("🤖 Chatbot com LangChain")
7
8 llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.7)
9
10 # Histórico de mensagens persistido na sessão do Streamlit
11 if "historico" not in st.session_state:
12     st.session_state.historico = []
13
14 # Exibir histórico
15 for msg in st.session_state.historico:
16     role = "user" if isinstance(msg, HumanMessage) else "assistant"
17     st.chat_message(role).write(msg.content)
18
19 # Input do usuário
20 if pergunta := st.chat_input("Digite sua pergunta:"):
21     st.session_state.historico.append(HumanMessage(content=pergunta))
22     st.chat_message("user").write(pergunta)
23
24     resposta = llm.invoke(st.session_state.historico)
25     st.session_state.historico.append(AIMessage(content=resposta.content))
26     st.chat_message("assistant").write(resposta.content)
```

APIs Rest

- Flask / FastAPI
- expor LLM como serviço

```
1 # Versão: fastapi>=0.110, uvicorn>=0.29, langchain-openai>=0.2
2 # api.py – execute com: uvicorn api:app --reload
3 from fastapi import FastAPI
4 from pydantic import BaseModel
5 from langchain_openai import ChatOpenAI
6
7 app = FastAPI(title="LangChain API")
8 llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
9
10 class Pergunta(BaseModel):
11     texto: str
12     temperatura: float = 0.0
13
14 class Resposta(BaseModel):
15     resposta: str
16     modelo: str
17
18 @app.post("/perguntar", response_model=Resposta)
19 async def perguntar(pergunta: Pergunta):
20     llm_custom = llm.configurable_fields(temperature=pergunta.temperatura)
21     resultado = llm_custom.invoke(pergunta.texto)
22     return Resposta(resposta=resultado.content, modelo="gpt-4o-mini")
23
24 # Teste via curl:
25 # curl -X POST http://localhost:8000/perguntar \
26 #     -H "Content-Type: application/json" \
27 #     -d '{"texto": "O que é LangChain?"}'
```


Instabilidades do Langchain

Gerenciando instabilidades: boas práticas concretas

```
# requirements.txt para essa aula
langchain>=0.3,<0.4
langchain-core>=0.3,<0.4
langchain-community>=0.3,<0.4
langchain-openai>=0.2,<0.3
langchain-groq>=0.1,<0.2
langchain-huggingface>=0.1,<0.2
faiss-cpu>=1.7
chromadb>=0.5
streamlit>=1.35
fastapi>=0.110
```



Fixar Versões

Use == ou >=X,<Y para evitar quebras em atualizações automáticas



Isolamento de Ambiente

Sempre utilize venv ou conda para isolar dependências por projeto



Documentação e Migração

Consulte docs da versão exata e o MIGRATION.md no GitHub

Instabilidades do Langchain

Lembrete Final Importante

Esta aula foca no **racional das integrações**.

Se um código quebrar no seu ambiente, o problema quase sempre é de **versão**, não de lógica.

Entendendo o racional, você será capaz de **adaptar o código** a qualquer versão futura.